INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	SISTEMAS DISTRIBUIDOS				
TÍTULO DE LA PRÁCTICA:	*Los Hilos (Threads)*				
NÚMERO DE PRÁCTICA:	*01*	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	2026A
FECHA DE PRESENTACIÓN	*17/04/2026*	**HORA DE PRESENTACIÓN**	23:59:00		
INTEGRANTE (s): GRUPO " 1 " • **Quispe Huaman, Rodrigo Ferdinand** • **Salas Aguilar Juan Victor** • **Huarca Pallani Jael Emerson** • **Chambi Orosco Luis Fernando**				**NOTA:**	
DOCENTE(s): *Mg. Maribel Molina Barriga*					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

Link github: https://github.com/salasvictor22/Laboratorio-Sistemas_Distribuidos-A.git

(Colocar la evidencia de solución de problemas, de los puntos I y II)

Caso Estudio: simular el proceso de cobro de un supermercado

En el presente informe se desarrolla una simulación del proceso de atención en un supermercado, donde una cajera procesa los productos de distintos clientes. El objetivo es analizar el comportamiento del programa en tres escenarios: ejecución secuencial, uso de hilos mediante la clase Thread y uso de hilos mediante la interfaz Runnable.

Caso 1: Ejecución secuencial (sin hilos)

En este primer caso se implementa una simulación del proceso de cobro en un supermercado utilizando un enfoque secuencial, es decir, sin el uso de hilos. Esto implica que las tareas se ejecutan una después de otra, sin superposición en el tiempo.

Cajera.java

La clase cajera representa a la entidad encargada de procesar las compras. El método procesarCompra() recibe un cliente y recorre su lista de productos, simulando el tiempo de procesamiento mediante pausas (Thread.sleep). Además, se calcula el tiempo transcurrido desde el inicio para mostrar el progreso. Todo este proceso se ejecuta en un único flujo, sin concurrencia.

Código:

Cajera.java ● | Extensión: One Dark Pro | Cliente.java | Main.java

Cajera.java > ...

```java
public class Cajera {

    private String nombre;
    public Cajera() {
    }

    public Cajera(String nombre) {
        this.nombre = nombre;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    // Método principal que procesa la compra de un cliente
    public void procesarCompra(Cliente cliente, long timeStamp) {

        System.out.println("La cajera " + this.nombre +
                " COMIENZA A PROCESAR LA COMPRA DEL CLIENTE " + cliente.getNombre() +
                " EN EL TIEMPO: " + (System.currentTimeMillis() - timeStamp) / 1000 + " seg");

        // Se recorren todos los productos del cliente
        for (int i = 0; i < cliente.getCarroCompra().length; i++) {

            // Simula el tiempo que tarda en procesar cada producto
            this.esperarXsegundos(cliente.getCarroCompra()[i]);

            // Muestra el tiempo en que se procesó cada producto
            System.out.println("Procesado el producto " + (i + 1) +
                    " -> Tiempo: " + (System.currentTimeMillis() - timeStamp) / 1000 + " seg");
        }

        System.out.println("La cajera " + this.nombre +
                " HA TERMINADO DE PROCESAR " + cliente.getNombre() +
                " EN EL TIEMPO: " + (System.currentTimeMillis() - timeStamp) / 1000 + " seg");
    }

    // Método que simula la espera en segundos
    private void esperarXsegundos(int segundos) {
        try {
            Thread.sleep(segundos * 1000);
        } catch (InterruptedException ex) {
            Thread.currentThread().interrupt();open
        }
    }
}
```

Cliente.java

La clase Cliente almacena la información de cada cliente, incluyendo su nombre y un arreglo de enteros que representa el tiempo que toma procesar cada producto. Esto permite simular diferentes cargas de compra para cada cliente.

Código:

Cajera.java | Cliente.java | Main.java

Cliente.java > Cliente

```java
public class Cliente {

    private String nombre;
    private int[] carroCompra;

    public Cliente() {
    }

    public Cliente(String nombre, int[] carroCompra) {
        this.nombre = nombre;
        this.carroCompra = carroCompra;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int[] getCarroCompra() {
        return carroCompra;
    }

    public void setCarroCompra(int[] carroCompra) {
        this.carroCompra = carroCompra;
    }
}
```

Main.java

En la clase Main se crean dos clientes y dos cajeras. Sin embargo, aunque existen dos cajeras, las llamadas al método procesarCompra() se realizan de forma secuencial. Es decir, primero se procesa completamente la compra del cliente 1 y luego la del cliente 2. No se utilizan hilos, por lo que no hay ejecución simultánea.

Código:

```java
Main.java > Main
public class Main {

    Run | Debug
    public static void main(String[] args) {

        Cliente cliente1 = new Cliente(nombre: "Cliente 1", new int[] { 2, 2, 1, 5, 2, 3 });
        Cliente cliente2 = new Cliente(nombre: "Cliente 2", new int[] { 1, 3, 5, 1, 1 });

        Cajera cajera1 = new Cajera(nombre: "Cajera 1");
        Cajera cajera2 = new Cajera(nombre: "Cajera 2");

        // Tiempo inicial de referencia
        long initialTime = System.currentTimeMillis();

        cajera1.procesarCompra(cliente1, initialTime);
        cajera2.procesarCompra(cliente2, initialTime);
    }
}
```

Evaluación de resultados

```
PS D:\SEPTIMO SEMESTRE\Laboratorio Sistemas Distribuidos> java Main
La cajera Cajera 1 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 1 EN EL TIEMPO: 0 seg
Procesado el producto 1 -> Tiempo: 2 seg
Procesado el producto 2 -> Tiempo: 4 seg
Procesado el producto 3 -> Tiempo: 5 seg
Procesado el producto 4 -> Tiempo: 10 seg
Procesado el producto 5 -> Tiempo: 12 seg
Procesado el producto 6 -> Tiempo: 15 seg
La cajera Cajera 1 HA TERMINADO DE PROCESAR Cliente 1 EN EL TIEMPO: 15 seg
La cajera Cajera 2 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 2 EN EL TIEMPO: 15 seg
Procesado el producto 1 -> Tiempo: 16 seg
Procesado el producto 2 -> Tiempo: 19 seg
Procesado el producto 3 -> Tiempo: 24 seg
Procesado el producto 4 -> Tiempo: 25 seg
Procesado el producto 5 -> Tiempo: 26 seg
La cajera Cajera 2 HA TERMINADO DE PROCESAR Cliente 2 EN EL TIEMPO: 26 seg
```

A partir de la ejecución del programa en su versión secuencial, se observa que la atención de los clientes se realiza de manera estrictamente ordenada, es decir, primero se procesa completamente la compra del cliente 1 y, una vez finalizada, se inicia la atención del cliente 2.

Según los resultados obtenidos, la cajera 1 termina de procesar al cliente 1 en aproximadamente 15 segundos, y recién en ese momento la cajera 2 comienza a atender al cliente 2, finalizando en el segundo 26. Esto evidencia que el tiempo total de ejecución corresponde a la suma de los tiempos individuales de cada cliente.

Este comportamiento confirma que el sistema no presenta concurrencia, ya que no existe superposición en la ejecución de tareas. Aunque se definen dos cajeras, estas no trabajan de manera simultánea, sino secuencial, lo que limita el aprovechamiento de los recursos del sistema.

Caso 2: Uso de hilos mediante la clase Thread

En este segundo caso se implementa la programación concurrente utilizando hilos, con el objetivo de procesar múltiples clientes de manera simultánea. A diferencia del caso anterior, donde la ejecución era secuencial, aquí cada cajera se ejecuta como un hilo independiente, permitiendo reducir el tiempo total de procesamiento.

CajeraTherad.java

La clase CajeraThread hereda de la clase Thread, lo que permite crear hilos de ejecución independientes. El método run() contiene la lógica que se ejecutará en cada hilo, es decir, el procesamiento de la compra de un cliente. Cada instancia de esta clase representa una cajera trabajando de forma concurrente, procesando productos de manera independiente.

Código:

CajeraThread.java

```java
public class CajeraThread extends Thread {

    private String nombre;
    private Cliente cliente;
    private long initialTime;

    public CajeraThread() {
    }

    // Constructor que inicializa la cajera, el cliente y el tiempo inicial
    public CajeraThread(String nombre, Cliente cliente, long initialTime) {
        this.nombre = nombre;
        this.cliente = cliente;
        this.initialTime = initialTime;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public long getInitialTime() {
        return initialTime;
    }

    public void setInitialTime(long initialTime) {
        this.initialTime = initialTime;
    }

    public Cliente getCliente() {
        return cliente;
    }

    public void setCliente(Cliente cliente) {
        this.cliente = cliente;
    }
```

```java

    // Método que se ejecuta al iniciar el hilo
    @Override
    public void run() {

        // Inicio del procesamiento de la compra
        System.out.println("La cajera " + this.nombre + " COMIENZA A PROCESAR LA COMPRA DEL CLIENTE "
                + this.cliente.getNombre() + " EN EL TIEMPO: "
                + (System.currentTimeMillis() - this.initialTime) / 1000
                + "seg");

        // Recorrido de los productos del cliente
        for (int i = 0; i < this.cliente.getCarroCompra().length; i++) {

            // Se procesa cada producto simulando el tiempo con sleep
            this.esperarXsegundos(cliente.getCarroCompra()[i]);

            // Mensaje de producto procesado con tiempo transcurrido
            System.out.println("Procesado el producto " + (i + 1)
                    + " del cliente " + this.cliente.getNombre() + "->Tiempo: "
                    + (System.currentTimeMillis() - this.initialTime) / 1000
                    + "seg");

        }

        // Fin del procesamiento del cliente
        System.out.println("La cajera " + this.nombre + " HA TERMINADO DE PROCESAR "
                + this.cliente.getNombre() + " EN EL TIEMPO: "
                + (System.currentTimeMillis() - this.initialTime) / 1000
                + "seg");

    }

    // Método para simular el tiempo de espera por producto
    private void esperarXsegundos(int segundos) {
        try {
            Thread.sleep(segundos * 1000);
        } catch (InterruptedException ex) {
            Thread.currentThread().interrupt();
        }
    }
}
```

MainThread.java

En la clase MainThread se crean dos objetos de tipo CajeraThread, cada uno asociado a un cliente distinto. Al invocar el método start(), se inicia la ejecución de cada hilo, lo que permite que ambas cajeras trabajen al mismo tiempo. Esto introduce concurrencia en el sistema, a diferencia del caso secuencial.

Código:

```java
CajeraThread.java    MainThread.java    Cliente.java
MainThread.java > MainThread
public class MainThread {

    Run | Debug
    public static void main(String[] args) {

        Cliente cliente1 = new Cliente(nombre: "Cliente 1", new int[] { 2, 2, 1, 5, 2, 3 });
        Cliente cliente2 = new Cliente(nombre: "Cliente 2", new int[] { 1, 3, 5, 1, 1 });

        // Tiempo inicial de referencia para medir duración del proceso
        long initialTime = System.currentTimeMillis();

        // Creación de hilos (cajeras) que procesarán a cada cliente
        CajeraThread cajera1 = new CajeraThread(nombre: "Cajera 1", cliente1, initialTime);
        CajeraThread cajera2 = new CajeraThread(nombre: "Cajera 2", cliente2, initialTime);

        cajera1.start();
        cajera2.start();
    }
}
```

Evaluación de resultados

```
PS D:\SEPTIMO SEMESTRE\lab SD GRUPO A\CASO 2> java MainThread
La cajera Cajera 2 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 2 EN EL TIEMPO: 0seg
La cajera Cajera 1 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 1 EN EL TIEMPO: 0seg
Procesado el producto 1 del cliente Cliente 2->Tiempo: 1seg
Procesado el producto 1 del cliente Cliente 1->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1->Tiempo: 4seg
Procesado el producto 2 del cliente Cliente 2->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1->Tiempo: 5seg
Procesado el producto 3 del cliente Cliente 2->Tiempo: 9seg
Procesado el producto 4 del cliente Cliente 1->Tiempo: 10seg
Procesado el producto 4 del cliente Cliente 2->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 2->Tiempo: 11seg
La cajera Cajera 2 HA TERMINADO DE PROCESAR Cliente 2 EN EL TIEMPO: 11seg
Procesado el producto 5 del cliente Cliente 1->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1->Tiempo: 15seg
La cajera Cajera 1 HA TERMINADO DE PROCESAR Cliente 1 EN EL TIEMPO: 15seg
```

Los resultados muestran que ambas cajeras inician el procesamiento al mismo tiempo, lo que confirma la ejecución concurrente mediante hilos. Los mensajes aparecen intercalados, evidenciando que los dos clientes son atendidos simultáneamente.

El tiempo total de ejecución es de aproximadamente 15 segundos, determinado por el cliente que tarda más en procesarse. Esto representa una mejora significativa frente al modelo secuencial (26 segundos), demostrando que el uso de hilos optimiza el rendimiento del sistema.

Caso 3: Uso de hilos mediante la interfaz Runnable

En este tercer caso se implementa la concurrencia utilizando la interfaz Runnable, lo cual permite ejecutar múltiples tareas en paralelo sin necesidad de heredar de la clase Thread. Este enfoque es más flexible y recomendado en Java, ya que separa la lógica de la tarea de la gestión del hilo.

MainRunnable.java

La clase MainRunnable implementa la interfaz Runnable, lo que permite definir la tarea que se ejecutará en un hilo mediante el método run(). En el método main, se crean dos procesos (uno por cada cliente) y se ejecutan en paralelo mediante objetos Thread. Este enfoque reutiliza la clase cajera del caso anterior y permite lograr concurrencia sin necesidad de heredar de Thread.

Código:

MainRunnable.java × Cliente.java Cajera.java

MainRunnable.java > MainRunnable

```java
public class MainRunnable implements Runnable {

    private Cliente cliente;
    private Cajera cajera;
    private long initialTime;

    // Constructor: recibe los datos necesarios para ejecutar el proceso en paralelo
    public MainRunnable(Cliente cliente, Cajera cajera, long initialTime) {
        this.cajera = cajera;
        this.cliente = cliente;
        this.initialTime = initialTime;
    }
    Run | Debug
    public static void main(String[] args) {

        Cliente cliente1 = new Cliente(nombre: "Cliente 1", new int[] { 2, 2, 1, 5, 2, 3 });
        Cliente cliente2 = new Cliente(nombre: "Cliente 2", new int[] { 1, 3, 5, 1, 1 });

        Cajera cajera1 = new Cajera(nombre: "Cajera 1");
        Cajera cajera2 = new Cajera(nombre: "Cajera 2");

        // Tiempo inicial para medir cuánto tarda todo el proceso
        long initialTime = System.currentTimeMillis();

        // Se crean los procesos (Runnable) para cada cliente
        Runnable proceso1 = new MainRunnable(cliente1, cajera1, initialTime);
        Runnable proceso2 = new MainRunnable(cliente2, cajera2, initialTime);

        // Se ejecutan en paralelo usando hilos (multithreading)
        new Thread(proceso1).start();
        new Thread(proceso2).start();
    }
    // Método que se ejecuta cuando inicia el hilo
    @Override
    public void run() {

        this.cajera.procesarCompra(this.cliente, this.initialTime);
    }
}
```

Evaluación de resultados

```
PS D:\SEPTIMO SEMESTRE\lab SD GRUPO A\CASO 3> java MainRunnable
La cajera Cajera 1 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 1 EN EL TIEMPO: 0 seg
La cajera Cajera 2 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 2 EN EL TIEMPO: 0 seg
Procesado el producto 1 -> Tiempo: 1 seg
Procesado el producto 1 -> Tiempo: 2 seg
Procesado el producto 2 -> Tiempo: 4 seg
Procesado el producto 2 -> Tiempo: 4 seg
Procesado el producto 3 -> Tiempo: 5 seg
Procesado el producto 3 -> Tiempo: 9 seg
Procesado el producto 4 -> Tiempo: 10 seg
Procesado el producto 4 -> Tiempo: 10 seg
Procesado el producto 5 -> Tiempo: 11 seg
La cajera Cajera 2 HA TERMINADO DE PROCESAR Cliente 2 EN EL TIEMPO: 11 seg
Procesado el producto 5 -> Tiempo: 12 seg
Procesado el producto 6 -> Tiempo: 15 seg
La cajera Cajera 1 HA TERMINADO DE PROCESAR Cliente 1 EN EL TIEMPO: 15 seg
```

Los resultados muestran que ambas cajeras inician el procesamiento al mismo tiempo, lo que confirma la ejecución concurrente mediante hilos utilizando la interfaz Runnable. La salida intercalada evidencia que los productos de ambos clientes se procesan de manera simultánea.

El tiempo total de ejecución es de aproximadamente 15 segundos, determinado por el cliente con mayor carga de productos. En conclusión, el uso de Runnable permite lograr concurrencia de manera eficiente y con un diseño más flexible, manteniendo los mismos beneficios de ejecución paralela que el uso de la clase Thread.

Conclusión

Tras analizar los tres casos, se puede concluir que el uso de hilos permite mejorar significativamente el rendimiento del sistema. En el modelo secuencial, el procesamiento se realiza de manera ordenada, lo que genera un mayor tiempo de ejecución debido a que cada cliente es atendido uno después de otro.

En cambio, tanto el uso de la clase Thread como la implementación de la interfaz Runnable permiten ejecutar múltiples tareas de forma concurrente, reduciendo el tiempo total aproximadamente a la mitad. Esto demuestra un mejor aprovechamiento de los recursos del sistema.

Finalmente, aunque ambos enfoques multihilo ofrecen resultados similares en rendimiento, la implementación mediante Runnable es más recomendable, ya que proporciona mayor flexibilidad y permite una mejor organización del código, siendo una práctica más adecuada en el desarrollo de aplicaciones concurrentes.

(Colocar la evidencia de solución de problemas propuestos punto III)

a. Implementar, ejecutar y probar el código de los anexos 2, de manera adecuada.

b. Evaluar resultados obtenidos.

c. Escriba un reporte sobre las tareas realizadas y resultados.

d. Escriba sus conclusiones

Caso Estudio: Productor - Consumidor

El productor genera un entero entre 0 y 9 (inclusive), lo almacena en un objeto "CubbyHole", e imprime el número generado. Para hacer más interesante el problema de la sincronización, el productor duerme durante un tiempo aleatorio entre 0 y 100 milisegundos antes de repetir el ciclo de generación de números.

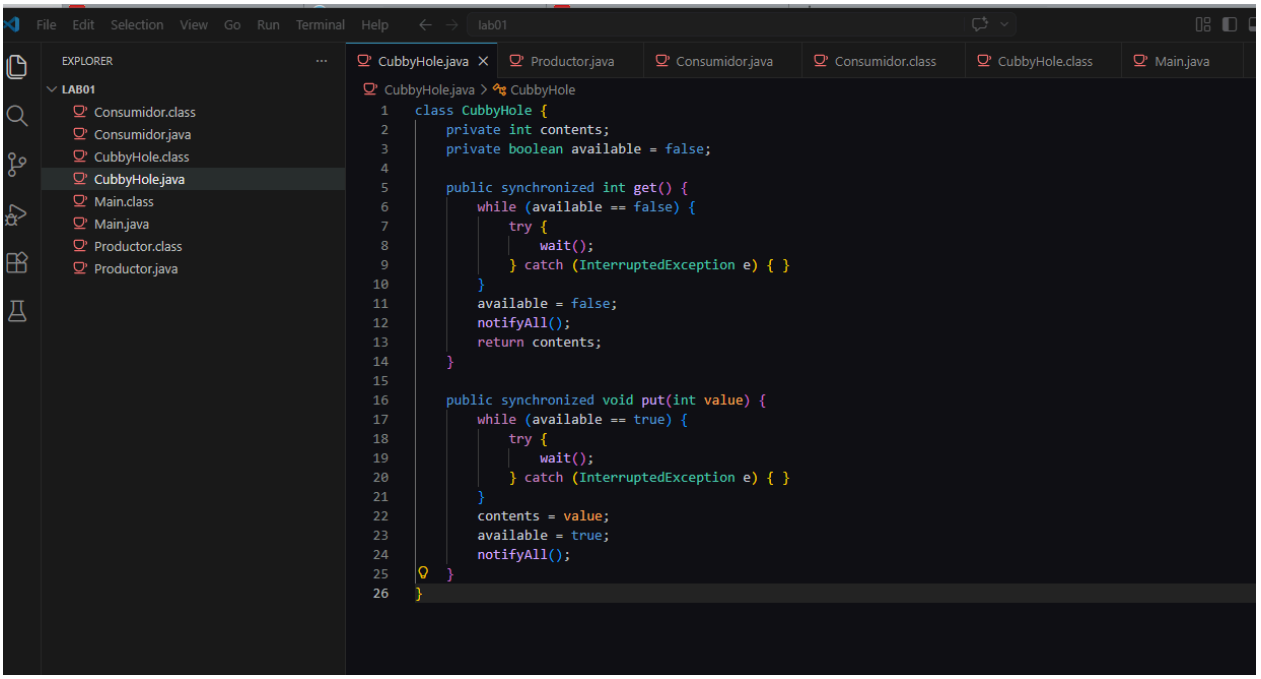
Código:

Consumidor.java

Es un hilo que obtiene los datos almacenados en el CubbyHole mediante el método get(). Consume los valores generados por el productor y los muestra en pantalla. Este hilo depende del productor, ya que solo puede consumir cuando existe un dato disponible.

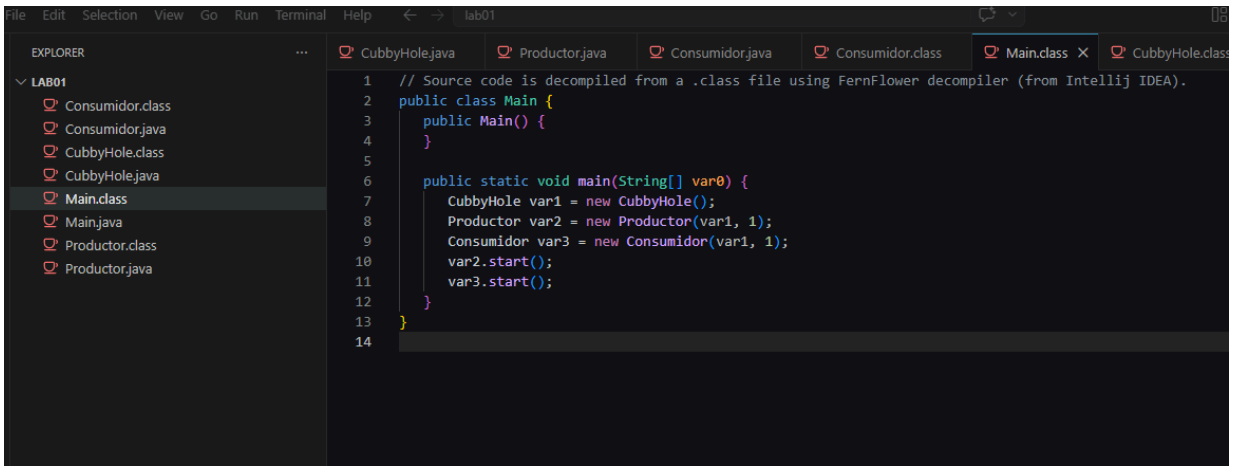
CubbyHole.java

Es el recurso compartido entre el productor y el consumidor. Funciona como un buffer de un solo espacio donde se almacena un dato. Utiliza métodos sincronizados (synchronized) para asegurar que solo un hilo acceda a la vez. Además, emplea wait() para pausar la ejecución cuando no se puede continuar (por ejemplo, cuando no hay datos o el espacio está ocupado) y notifyAll() para reactivar los hilos en espera. Su función principal es garantizar una correcta coordinación entre los hilos.



Main.java

Es la clase principal del programa. Se encarga de crear el objeto compartido (CubbyHole), instanciar los hilos Productor y Consumidor, y ejecutar ambos mediante el método start(). Aquí se inicia la ejecución concurrente.



Productor.java

Es un hilo encargado de generar números del 0 al 9. Cada número es enviado al CubbyHole mediante el método put(). Después de cada producción, el hilo duerme un tiempo aleatorio entre 0 y 100 ms, lo que permite simular un entorno concurrente donde los procesos no siguen un orden fijo.

```java
class Productor extends Thread {
    private CubbyHole cubbyhole;
    private int numero;

    public Productor(CubbyHole c, int numero) {
        cubbyhole = c;
        this.numero = numero;
    }

    public void run() {
        for (int i = 0; i < 10; i++) {
            cubbyhole.put(i);
            System.out.println("Productor #" + this.numero + " pone: " + i);

            try {
                sleep((int)(Math.random() * 100));
            } catch (InterruptedException e) { }
        }
    }
}
```

Evaluación de Resultados

```
OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    PROBLEMS

PS C:\Users\luis_\Desktop\Distribuidos lab\lab01> java Main
Productor #1 pone: 0
Consumidor #1 obtiene: 0
Productor #1 pone: 1
Consumidor #1 obtiene: 1
Productor #1 pone: 2
Consumidor #1 obtiene: 2
Consumidor #1 obtiene: 3
Productor #1 pone: 3
Productor #1 pone: 4
Consumidor #1 obtiene: 4
Consumidor #1 obtiene: 5
Productor #1 pone: 5
Productor #1 pone: 6
Consumidor #1 obtiene: 6
Productor #1 pone: 7
Consumidor #1 obtiene: 7
Consumidor #1 obtiene: 8
Productor #1 pone: 8
Productor #1 pone: 9
Consumidor #1 obtiene: 9
PS C:\Users\luis_\Desktop\Distribuidos lab\lab01>
```

Al ejecutar el programa del problema Productor–Consumidor, se obtuvo una salida donde el productor genera números del 0 al 9 y el consumidor los obtiene correctamente.

Se verificó que:

- Se produjeron exactamente 10 valores.
- Cada valor fue consumido una sola vez.
- No se presentaron datos duplicados ni pérdida de información.

Durante la ejecución, se observó que en algunos casos el consumidor imprime un valor antes que el productor. Esto no representa un error, sino que es consecuencia de la ejecución concurrente de los hilos, donde el orden de impresión puede variar.

En general, los resultados obtenidos demuestran que la sincronización implementada mediante synchronized, wait() y notifyAll() funciona correctamente, permitiendo una adecuada coordinación entre el productor y el consumidor y evitando condiciones de carrera.

Este comportamiento evidencia cómo es posible coordinar múltiples hilos que comparten un mismo recurso sin generar inconsistencias, asegurando que cada dato producido sea procesado una sola vez y de forma segura dentro del sistema.

Conclusiones:

El problema Productor–Consumidor evidencia la importancia de la sincronización en entornos concurrentes, ya que permite una correcta interacción entre hilos que comparten un mismo recurso, evitando conflictos en el acceso a los datos.

El uso de mecanismos como synchronized, wait() y notifyAll() es fundamental para prevenir errores como duplicación o pérdida de información, asegurando que cada dato sea consumido una sola vez.

Finalmente, aunque los hilos se ejecutan sin un orden fijo, la sincronización garantiza resultados consistentes y confiables, evitando condiciones de carrera.

Caso Estudio 2 del productor-consumidor modificado

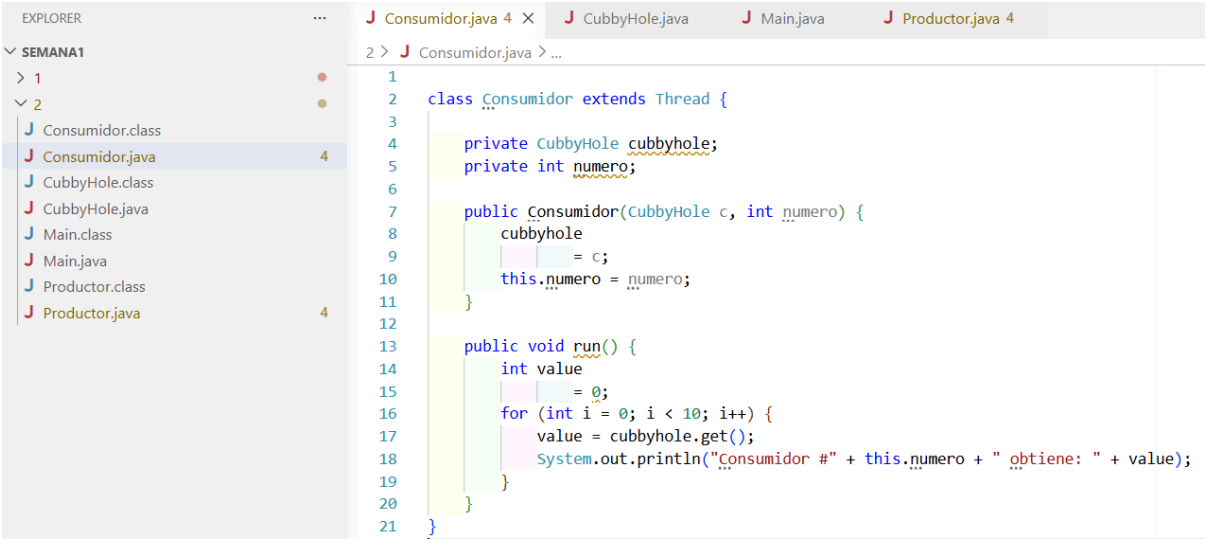
En este caso se presenta una modificación del problema clásico Productor–Consumidor, haciendo énfasis en el uso de monitores en Java como mecanismo de sincronización. A diferencia del caso anterior, aquí se resalta cómo Java gestiona automáticamente la exclusión mutua mediante monitores asociados a cada objeto.

El productor genera valores enteros y los almacena en el objeto compartido CubbyHole, mientras que el consumidor los recupera. La coordinación entre ambos hilos se realiza mediante métodos sincronizados y el uso de wait() y notify(), apoyándose en el monitor del objeto.

Código:

Consumidor.java

Es un hilo que obtiene los datos almacenados en el CubbyHole mediante el método get(). En este caso, el consumidor adquiere el monitor del objeto al ingresar al método sincronizado. Si no hay datos disponibles, el hilo entra en estado de espera mediante wait(), liberando temporalmente el monitor hasta que el productor genere un nuevo valor. Una vez disponible, consume el dato y lo muestra en pantalla.



CubbyHole.java

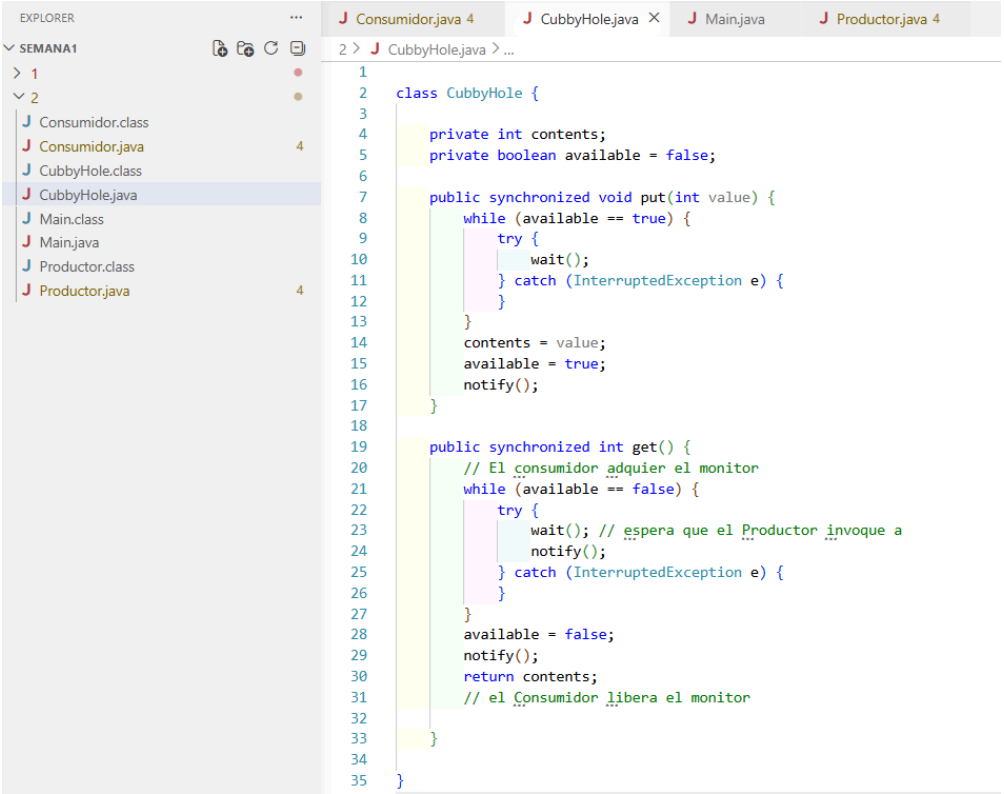
Es el recurso compartido que implementa el concepto de monitor. Contiene una variable contents que almacena el valor actual y una variable booleana available que indica si el dato está disponible para ser consumido.

Los métodos put() y get() son sincronizados, lo que implica que solo un hilo puede ejecutarlos a la vez, ya que al entrar en ellos el hilo adquiere el monitor del objeto.

En put(), el productor espera mientras el espacio esté ocupado (available == true).
En get(), el consumidor espera mientras no haya datos disponibles (available == false).

El uso de wait() permite liberar el monitor temporalmente, mientras que notify() despierta a los hilos en espera. Este mecanismo asegura una correcta coordinación entre productor y consumidor sin conflictos de acceso.

Además, este caso evidencia el uso de monitores reentrantes en Java, donde un hilo puede adquirir nuevamente un monitor que ya posee sin generar bloqueos.

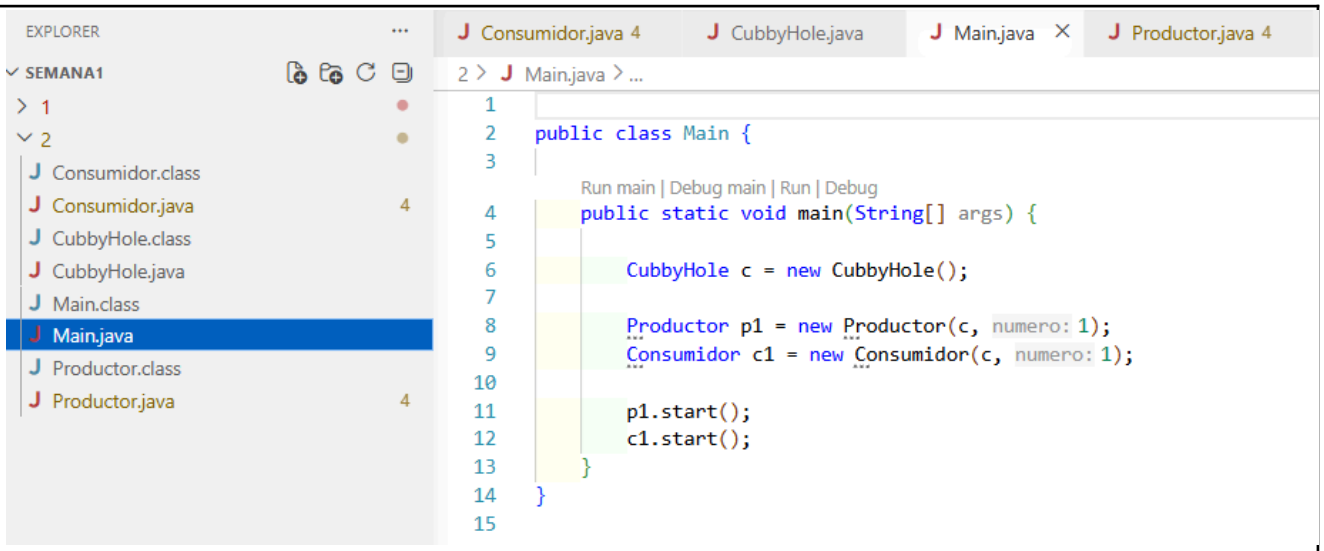



```java
class CubbyHole {

    private int contents;
    private boolean available = false;

    public synchronized void put(int value) {
        while (available == true) {
            try {
                wait();
            } catch (InterruptedException e) {
            }
        }
        contents = value;
        available = true;
        notify();
    }

    public synchronized int get() {
        // El consumidor adquier el monitor
        while (available == false) {
            try {
                wait(); // espera que el Productor invoque a
                notify();
            } catch (InterruptedException e) {
            }
        }
        available = false;
        notify();
        return contents;
        // el Consumidor libera el monitor

    }

}
```

Main.java

Es la clase principal del programa. Se encarga de crear la instancia del objeto compartido (CubbyHole) y de iniciar los hilos Productor y Consumidor mediante el método start().

Aquí se da inicio a la ejecución concurrente, donde ambos hilos interactúan utilizando el monitor del objeto compartido para sincronizar sus operaciones.



Productor.java

Es un hilo encargado de generar valores enteros y almacenarlos en el CubbyHole mediante el método put().

Al ingresar al método sincronizado, el productor adquiere el monitor del objeto, impidiendo que el consumidor acceda simultáneamente. Si el espacio está ocupado, el productor entra en espera con wait(). Una vez que el consumidor libera el recurso, el productor continúa su ejecución.

Este comportamiento permite simular un entorno concurrente controlado, donde la producción de datos depende del estado del recurso compartido.

EXPLORER

SEMANA1

- 1
- 2
  - Consumidor.class
  - Consumidor.java 4
  - CubbyHole.class
  - CubbyHole.java
  - Main.class
  - Main.java
  - Productor.class
  - Productor.java 4

Consumidor.java 4 | CubbyHole.java | Main.java | Productor.java 4

2 > Productor.java > ...

```java
class Productor extends Thread {

    private CubbyHole cubbyhole;
    private int numero;

    public Productor(CubbyHole c, int numero) {
        cubbyhole = c;
        this.numero
                = numero;
    }

    public void run() {
        for (int i = 0; i < 10;
                i++) {
            cubbyhole.put(i);
            System.out.println("Productor #" + this.numero + "pone:" + i);
            try {
                sleep((int) (Math.random() * 100));
            } catch (InterruptedException e) {
            }

        }
    }
}
```

EVALUACIÓN DE RESULTADOS

PowerShell 7 (x64)

```
PowerShell 7.6.0
Loading personal and system profiles took 1019ms.
PS C:\Users\Rodrigo> cd C:\UNSA\7SEMESTRE\SD_LAB\SEMANA1\2
PS C:\UNSA\7SEMESTRE\SD_LAB\SEMANA1\2> javac *.java
PS C:\UNSA\7SEMESTRE\SD_LAB\SEMANA1\2> java Main
Productor #1pone:0
Consumidor #1 obtiene: 0
Productor #1pone:1
Consumidor #1 obtiene: 1
Productor #1pone:2
Consumidor #1 obtiene: 2
Productor #1pone:3
Consumidor #1 obtiene: 3
Productor #1pone:4
Consumidor #1 obtiene: 4
Productor #1pone:5
Consumidor #1 obtiene: 5
Productor #1pone:6
Consumidor #1 obtiene: 6
Productor #1pone:7
Consumidor #1 obtiene: 7
Productor #1pone:8
Consumidor #1 obtiene: 8
Productor #1pone:9
Consumidor #1 obtiene: 9
PS C:\UNSA\7SEMESTRE\SD_LAB\SEMANA1\2>
```

Al ejecutar el programa modificado del problema Productor–Consumidor utilizando monitores, se obtuvo una salida correcta donde el productor genera valores y el consumidor los procesa adecuadamente.

Se verificó que:

Los valores fueron producidos y consumidos de manera ordenada.
No se presentaron inconsistencias en los datos compartidos.
Cada valor producido fue consumido una sola vez.

Durante la ejecución, se observó que los hilos no siguen un orden estrictamente secuencial, lo cual es normal en sistemas concurrentes. Sin embargo, gracias al uso de monitores, se garantiza que no haya acceso simultáneo al recurso compartido.

El comportamiento observado demuestra que el uso de métodos sincronizados junto con wait() y notify() permite una correcta gestión del monitor, evitando condiciones de carrera y asegurando la integridad de los datos.

En general, los resultados confirman que el mecanismo de monitores en Java proporciona una forma eficiente y segura de coordinar la ejecución de múltiples hilos que comparten un mismo recurso.

Conclusiones

El uso de monitores en este caso permite gestionar la sincronización de forma más estructurada, ya que cada objeto controla automáticamente el acceso mediante métodos sincronizados.

A diferencia del caso anterior, aquí se evidencia con mayor claridad cómo el monitor del objeto regula la ejecución de los hilos, permitiendo que solo uno acceda al recurso compartido mientras los demás esperan.

El uso de wait() y notify() dentro del monitor garantiza una correcta comunicación entre productor y consumidor, evitando accesos indebidos y asegurando la consistencia de los datos.

II. SOLUCIÓN DEL CUESTIONARIO

a. ¿Por qué es importante el estudio de hilos y multihilos en un sistema distribuido?

El estudio de los hilos (threads) y el multihilo es fundamental en los sistemas distribuidos porque estos sistemas están diseñados para ejecutar múltiples tareas de manera simultánea en diferentes nodos o computadoras conectadas en red.

En primer lugar, los hilos permiten la concurrencia, lo que posibilita que varias tareas se ejecuten de manera simultánea o intercalada dentro de un mismo proceso. Esto es esencial en un sistema

distribuido, donde múltiples usuarios o servicios realizan solicitudes al mismo tiempo, como ocurre en servidores web, sistemas en la nube o aplicaciones cliente-servidor.

Asimismo, el uso de multihilos mejora significativamente el rendimiento, ya que permite aprovechar mejor los procesadores multinúcleo y reducir los tiempos de espera. Mientras un hilo se encuentra bloqueado (por ejemplo, esperando una respuesta de red), otros pueden continuar ejecutándose, optimizando así el uso del CPU.

Otro aspecto importante es la eficiencia en el uso de recursos. Los hilos consumen menos memoria y tiempo de creación que los procesos, lo que permite manejar más tareas con menor costo computacional. También contribuyen a la escalabilidad, una característica clave en sistemas distribuidos. Gracias a los hilos, un sistema puede crecer y atender más usuarios o procesos sin perder rendimiento.

Finalmente, los hilos facilitan la implementación de aplicaciones distribuidas modernas, como microservicios, servidores concurrentes y procesamiento en paralelo, los cuales son esenciales en entornos como cloud computing y big data.

b. Describe ¿Cómo están compuestos los hilos? y ¿Cuál es la diferencia entre hilos y procesos?

¿Cómo están compuestos los hilos?

Un hilo es la unidad básica de ejecución dentro de un proceso y está compuesto por elementos esenciales que le permiten ejecutarse de manera independiente dentro del mismo programa.

Cada hilo incluye:

- **Contador de programa (Program Counter)**: indica la instrucción que se ejecutará a continuación.
- **Registros del CPU**: almacenan información temporal necesaria para la ejecución.
- **Pila (stack)**: contiene variables locales, parámetros de funciones y direcciones de retorno.
- **Estado del hilo**: representa su situación actual (ejecución, listo, bloqueado, etc.).

A diferencia de los procesos, los hilos comparten recursos del proceso al que pertenecen, como la memoria, el código y los archivos abiertos, lo que permite una ejecución más eficiente.

Diferencia entre hilos y procesos

Los hilos y los procesos presentan varias diferencias importantes en cuanto a su funcionamiento, uso de recursos y forma de ejecución dentro de un sistema. Mientras que los hilos permiten realizar múltiples tareas dentro de un mismo proceso de manera eficiente, los procesos funcionan como unidades independientes con mayor aislamiento. A continuación, se muestra una tabla que resume las principales diferencias entre ambos conceptos.

Característica	Hilos (Threads)	Procesos
Definición	Unidad de ejecución dentro de un proceso	Programa en ejecución independiente
Memoria	Comparten memoria del proceso	Tienen memoria propia
Comunicación	Rápida (memoria compartida)	Más lenta (IPC: pipes, sockets, etc.)
Creación	Rápida y de bajo costo	Lenta y costosa
Consumo de recursos	Bajo	Alto
Independencia	Dependen del proceso	Totalmente independientes
Seguridad	Menor (comparten memoria)	Mayor aislamiento
Fallos	Un error puede afectar a todo el proceso	Un error no afecta a otros procesos
Uso típico	Tareas concurrentes dentro de una app	Ejecución de aplicaciones separadas

c. Realice un cuadro comparativo de ventajas y desventajas del uso de hilos

Ventajas	**Desventajas**
• Podemos hacer varias cosas al mismo tiempo. • Aprovecha mejor la computadora. • El programa puede ir más rápido. • Hace que las apps no se queden congeladas. • Sirve para dividir el trabajo en partes. • Es útil en cosas como descargas o procesos largos.	• Es más complicado de programar. • Pueden aparecer errores raros. • A veces se necesita controlar bien los hilos. • Puede usar más recursos si hay muchos hilos. • Es difícil encontrar errores cuando algo falla. • Puede haber problemas si varios hilos usan lo mismo.

II. CONCLUSIONES

A partir del desarrollo de la práctica, se pudo comprender la importancia del uso de hilos en la programación concurrente dentro de los sistemas distribuidos. En el caso secuencial, se observó que las tareas se ejecutan una tras otra, lo que genera un mayor tiempo de ejecución y un uso poco eficiente de los recursos del sistema.

En cambio, al utilizar hilos mediante la clase Thread y la interfaz Runnable, se logró ejecutar múltiples tareas de manera simultánea, reduciendo significativamente el tiempo total de procesamiento y mejorando el rendimiento del sistema.

Asimismo, en el caso del problema Productor–Consumidor, se evidenció la importancia de la sincronización entre hilos. Sin un control adecuado, pueden presentarse errores como valores repetidos o pérdida de datos, conocidos como condiciones de carrera.

Finalmente, el uso de mecanismos como synchronized, wait() y notify() permite coordinar correctamente los hilos, asegurando que los datos se procesen de manera ordenada y sin inconsistencias, lo cual es fundamental en aplicaciones concurrentes.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, Sistemas operativos, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
[2] A. S. Tanenbaum and M. Van Steen, Sistemas distribuidos, 3rd ed. Upper Saddle River, NJ, USA: Pearson Education, 2017.
[3] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.